

Rapport de TP - Day 1

Algorithmes de Base, Fichiers & CLI de Surveillance de Serveurs

Date de rendu :	29/06/2026 14:15
Etudiant :	Amine (M2)
Statut :	100% DES TESTS VALIDES

1. Lab 1 : Programmation Python de Base & Fichiers

Ce premier module valide les concepts fondamentaux de Python : boucles, manipulation de chaines de caracteres, listes de comprehension, ainsi que la lecture/ecriture de fichiers plats et JSON.

Exercice 1.1 : Calcul de la Factorielle (Boucle)

Objectif : Completer la fonction `factorial(n)` pour calculer la factorielle d'un nombre entier en utilisant une boucle.

```
def factorial(n):  
    """Compute the factorial of a number."""  
    result = 1  
    for i in range(1, n + 1): # <-- Rempli : n + 1  
        result *= i # <-- Rempli : i  
    return result
```

[OK] Test de validation :

```
factorial(5) -> 120  
factorial(3) -> 6  
factorial(1) -> 1  
All test cases passed!
```

Exercice 1.2 : Inversion de Chaine de Caracteres

Objectif : Retourner une chaine de caracteres inversee en utilisant le slicing Python.

```
def reverse_string(s):  
    """Reverse a string."""  
    return s[::-1] # <-- Rempli : s[::-1]
```

[OK] Test de validation :

```
reverse_string('Python') -> 'nohtyP'  
reverse_string('Data') -> 'ataD'  
All test cases passed!
```

Exercice 1.3 : Filtrer les Nombres Pairs

Objectif : Utiliser une liste de comprehension pour extraire uniquement les nombres pairs d'une liste.

```
def find_even_numbers(lst):  
    """Return a list of even numbers."""  
    return [num for num in lst if num % 2 == 0] # <-- Rempli : num et num % 2 == 0
```

[OK] Test de validation :

```
find_even_numbers([1, 2, 3, 4, 5, 6]) -> [2, 4, 6]  
find_even_numbers([10, 15, 20, 25]) -> [10, 20]  
All test cases passed!
```

Exercice 1.4 : Ecriture et Lecture dans un Fichier Texte

Objectif : Ecrire une liste de chaines dans un fichier avec des retours a la ligne, puis lire ce fichier pour reconstruire la liste d'origine.

```
# Writing to a file
def write_to_file(filename, data):
    with open(filename, 'w') as file: # <-- Rempli : 'w'
        for name in data:
            file.write(name + '\n') # <-- Rempli : write(name + '\n')

# Reading from a file
def read_from_file(filename):
    with open(filename, 'r') as file: # <-- Rempli : 'r'
        return file.readlines() # <-- Rempli : readlines()
```

[OK] Test de validation :

```
write_to_file('names.txt', ['Alice', 'Bob', 'Charlie'])
read_from_file('names.txt') -> ['Alice\n', 'Bob\n', 'Charlie\n']
All test cases passed!
```

Exercice 1.5 : Ecriture et Lecture JSON

Objectif : Utiliser le module json pour sauvegarder et charger un dictionnaire dans un fichier structure.

```
import json

# Writing to a JSON file
def write_json(filename, data):
    with open(filename, 'w') as file: # <-- Rempli : 'w'
        json.dump(data, file, indent=4) # <-- Rempli : data

# Reading from a JSON file
def read_json(filename):
    with open(filename, 'r') as file: # <-- Rempli : 'r'
        return json.load(file) # <-- Rempli : load(file)
```

[OK] Test de validation :

```
write_json('data.json', {...})
read_json('data.json') -> {...}
All test cases passed!
```

2. Lab 1.2 : Outil CLI de Surveillance des Serveurs

Ce lab consiste a developper un script complet de surveillance de serveurs. Il charge un fichier JSON contenant une liste de serveurs, verifie le statut de chacun a l'aide de requetes HTTP avec timeout, et genere un rapport consolide.

Etape 1 : Chargement de la Configuration (load_servers)

```
def load_servers(path: str) -> list[dict]:
    """Load and return the list of servers from a JSON file."""
    try:
        raw = pathlib.Path(path).read_text(encoding='utf-8')
        return json.loads(raw)
```

```
except FileNotFoundError:
    print(f"Error: The file '{path}' does not exist.", file=sys.stderr)
    sys.exit(1)
except json.JSONDecodeError:
    print(f"Error: The file '{path}' is not valid JSON.", file=sys.stderr)
    sys.exit(1)
```

Etape 2 : Verification Individuelle des Serveurs (check_server)

Cette etape utilise la bibliotheque requests. Les serveurs sont qualifies de :

- UP : Reponse HTTP 200 en moins de 500ms.
- DEGRADED : Reponse HTTP 200 mais superieure a 500ms, ou code HTTP different de 200.
- DOWN : Timeout de 5s atteint ou erreur de connexion.

```
def check_server(server: dict) -> dict:
    """Check the health of a single server and return a result dict."""
    url = f"{server['protocol']}://{server['host']}:{server['port']}{server['health_path']}"
    start = time.time()
    try:
        resp = requests.get(url, timeout=5)
        elapsed_ms = (time.time() - start) * 1000

        status_code = resp.status_code
        if status_code == 200:
            if elapsed_ms <= 500:
                status = "UP"
            else:
                status = "DEGRADED"
        else:
            status = "DEGRADED"

        return {
            "name": server["name"],
            "url": url,
            "status": status,
            "response_time_ms": elapsed_ms,
            "status_code": status_code
        }
    except (requests.exceptions.ConnectionError, requests.exceptions.Timeout) as e:
        return {
            "name": server["name"],
            "url": url,
            "status": "DOWN",
            "response_time_ms": None,
            "status_code": None,
            "error": str(e)
        }
```

Etape 3 : Rapport de Synthese (run_health_check)

```
def run_health_check(servers: list[dict]) -> dict:
    """Check all servers and return a summary report."""
    results = []
    up, degraded, down = 0, 0, 0
    for server in servers:
        res = check_server(server)
        results.append(res)
        if res["status"] == "DOWN":
            down += 1
        elif res["status"] == "UP":
            up += 1
        else:
```

```
        degraded += 1
    return {
        "checked_at": datetime.now().isoformat(timespec='seconds'),
        "total": len(servers),
        "up": up,
        "degraded": degraded,
        "down": down,
        "results": results
    }
```

Exemple de Rapport d'Execution (Console)

[OK] Test de validation :

```
[o_o] api-prod-1 : UP (HTTP 200, 145.2ms)
[o_o] api-prod-2 : DEGRADED (HTTP 503, 152.8ms)
[-_-] slow-server : DOWN (Error: Read timed out. (read timeout=5))
[-_-] unreachable : DOWN (Error: Connection to 10.255.255.1 timed out.)
```

Report Summary:

```
- Total: 4, UP: 1, DEGRADED: 1, DOWN: 2
- Checked at: 2026-06-29T14:07:30
```